

诚信声明

我声明，所呈交的毕业论文（设计）是本人在老师指导下进行的研究工作及取得的研究成果。除文中已经注明引用的内容外，本论文（设计）不包含任何其他个人或集体已经发表或撰写过的作品成果。在论文（设计）撰写过程中，本人严格遵守学术道德和学术规范，恪守学术诚信，坚决杜绝任何形式的抄袭、剽窃、伪造、篡改、买卖、代写毕业论文等学术不端行为，本人自愿承担因违反上述承诺而产生的后果和责任。

签名：

日期： 年 月 日

基于 Django 的用户选车推荐系统设计与实现

[摘要] 目前国内外的汽车市场对于买车消费者来说存在较多问题，例如在售车型种类繁多，现有的线上选车平台个性化功能欠缺以及消费者选车的决策成本高，导致用户选车决策周期长，难以选到适合车型。本文为解决消费者在面对选车购车时的选车需求，缩短用户选车决策周期，根据用户的需求，各类使用场景及偏好实现精准车辆推荐，设计并实现了基于 Django 的用户选车推荐系统。

该系统使用 python 开发语言，基于 Django 框架完成后端开发，使用 Bootstrap 的前端框架和 ECharts 库完成前端开发。该系统实现了个性化推荐，汽车信息查询，参数对比，汽车数据可视化，智能客服，数据采集等核心功能，能够为用户提供直观的可视化数据，提供信息查询补齐信息差，实现精准化个性化的智能推荐，具有实用意义。

[关键词] Django 框架；选车推荐；个性化推荐；汽车数据可视化

Design and Implementation of a Django-Based User Car Selection Recommendation System

Abstract: Currently, the domestic and international automobile markets present several challenges for car buyers. For instance, the wide variety of available models, the lack of personalized features on existing online car selection platforms, and the high decision-making costs for consumers lead to long car selection cycles, making it difficult for users to find suitable models. To address consumers' car selection needs, shorten their decision-making cycles, and achieve precise vehicle recommendations based on user requirements, usage scenarios, and preferences, this paper designs and implements a Django-based user car selection recommendation system.

The system is developed using the Python programming language, with the backend built on the Django framework and the frontend developed using the Bootstrap framework and the ECharts library. The system implements core functions such as personalized recommendations, car information queries, parameter comparisons, car data visualization, an intelligent customer service module, and data collection. It provides users with intuitive visual data, bridges information gaps through information queries, and delivers accurate and personalized intelligent recommendations, offering significant practical value.

Keywords : Django Framework; Car Selection Recommendation; Personalized Recommendation; Car Data Visualization

目 录

1 绪论	1
1.1 开发背景	1
1.2 国内外研究现状	1
1.3 目标及意义	2
1.4 本文的工作与内容安排	3
2 相关技术	4
2.1 后端	4
2.1.1 Django Web 框架	4
2.1.2 协同过滤推荐算法	4
2.1.3 网络爬虫技术	4
2.2 前端	5
2.2.1 Echarts	5
2.2.2 Bootstrap	5
3 概要设计	6
3.1 需求分析	6
3.1.1 管理员	6
3.1.2 用户	6
3.2 功能模块设计	7
4 数据库设计	9
4.1 数据库简介	9
4.2 概念结构设计	9
4.2.1 E-R 图	9
4.2.2 实体间联系图	10
4.3 逻辑结构设计	10
5 详细设计	15
5.1 数据分析模块	15
5.1.1 市场销量分析设计	15
5.1.2 小众场景分析设计	17
5.2 个性化推荐模块设计	18

5.3 智能客服模块设计.....	19
5.4 汽车对比模块设计.....	19
5.5 用户模块设计.....	20
5.5.1 汽车朋友圈.....	20
5.5.2 资料修改.....	21
5.6 数据采集模块设计.....	21
5.7 管理员模块设计.....	22
6 系统实现.....	23
6.1 市场销量分析实现.....	23
6.1.1 市场概览模块实现.....	23
6.1.2 价格分析模块实现.....	25
6.1.3 能源分析模块实现.....	25
6.1.4 品牌分析模块实现.....	26
6.2 小众场景分析实现.....	27
6.2.1 后端辅助函数设计.....	28
6.2.2 网约车场景实现.....	29
6.2.3 家庭用车场景实现.....	30
6.3 个性化推荐实现.....	30
6.3.1 用户评分向量构建.....	30
6.3.2 用户相似度计算.....	31
6.3.3 推荐结果生成.....	31
6.4 智能客服实现.....	31
6.5 汽车对比实现.....	32
6.5.1 评分机制.....	32
6.5.2 推荐结果生成.....	33
6.6 用户模块实现.....	33
6.6.1 个人信息修改.....	33
6.6.2 汽车朋友圈.....	34
6.7 数据采集实现.....	35
6.8 管理员模块实现.....	36
6.8.1 信息管理实现.....	37
6.8.2 社区管理实现.....	37
6.8.3 认证授权实现.....	37
7 系统功能演示.....	38
7.1 用户模块.....	38
7.1.1 登录与注册.....	38
7.1.2 资料修改.....	38
7.1.3 汽车朋友圈.....	39
7.2 数据分析.....	41

7.2.1 市场销量分析.....	41
7.2.2 小众场景分析.....	45
7.3 个性化推荐.....	46
7.4 智能客服.....	47
7.5 汽车对比.....	47
7.6 数据采集.....	52
7.7 管理员模块.....	54
参考文献.....	57
致谢	59

1 绪论

1.1 开发背景

随着我国汽车工业的快速发展，尤其是国产新能源汽车品牌的崛起，汽车消费市场呈现出车型多样化、配置差异化、用户需求个性化的显著特征。据统计，目前国内汽车市场在售车型已超千款，涵盖燃油车、纯电动车、插电混动、增程式等多类动力形式，价格区间、配置参数、适用场景等维度差异较大，给消费者选车带来了信息过载和决策困难的双重挑战。

当前消费者选车主要依赖两类渠道：一是线上垂直媒体平台，如汽车之家、懂车帝，通过参数对比、用户口碑、专业评测等获取信息。二是线下 4S 店，通过销售咨询与试驾体验获取直观感受。然而，现有渠道存在明显不足：线上平台信息繁多且分散，消费者需花费大量时间跨平台筛选比对，且推荐机制以热度或广告投放为导向，缺乏对用户个体需求的深度理解。线下渠道受地域和时间限制，销售推荐存在明显的利益偏向性，难以客观匹配消费者的真实用车场景。上述问题导致消费者选车决策周期长、满意度低，迫切需要一种能够融合多源信息、理解用户偏好、实现精准推荐的智能化选车辅助工具。

1.2 国内外研究现状

在汽车推荐领域，国内外学者与工业界已开展了相关探索，主要集中于以下两个方向。

(1) 基于内容与多属性决策的汽车推荐方法：针对汽车产品多维度参数的特点，部分研究引入多属性决策理论。徐畅等人提出了考虑期望强度与

属性补偿关系的新能源汽车在线推荐方法^[1],通过构建用户期望与车型属性之间的匹配模型,实现了在用户明确表达需求场景下的精准推荐。然而,该方法依赖于用户主动、准确地输入选车条件,对于需求模糊或缺乏汽车知识的用户,推荐效果受限。此外,张胜根等人构建了对标车型性能数据库^[2],为车型参数对比提供了数据基础,但该数据库主要面向专业分析场景,缺乏面向普通消费者的推荐能力。

(2) 工业界汽车选车平台的实践现状:在工业应用层面,汽车之家、懂车帝、CarGurus 等主流平台已集成初步的推荐功能,如猜你喜欢模块,主要基于热度排序、浏览历史关联或简单的规则筛选。这些功能在一定程度上缓解了信息过载问题,但推荐逻辑偏重商业转化,如广告车型优先展示,个性化程度有限,且缺乏对用户用车场景的深度建模能力。同时,现有平台普遍缺少直观的车型数据可视化工具,用户难以快速理解不同车型在多维度上的综合表现。

1.3 目标及意义

平台的开发与应用对消费者而言具有现实意义与参考价值,能够显著节省选车过程中的时间成本,规避信息不对称带来的决策风险,帮助其选择更贴合自身实际需求的车型。与此同时,从技术应用角度来看,平台融合了多源数据整合、个性化推荐算法设计与用户体验优化等核心技术,其开发实践可为同类消费决策类平台的研发提供可参考的技术路径与实现方案。

1.4 本文的工作与内容安排

本文基于用户需求调研与汽车消费行业现状分析，参考相关文献，以精准推荐、信息整合、便捷使用为核心设计理念，完成选车推荐平台的全流程开发工作。论文共分为七章，各章节具体内容安排如下：

第一章为绪论，主要介绍选车推荐平台的开发背景，分析现有消费者选车渠道存在的不足，明确项目的开发目标与现实意义，同时介绍本文的核心工作与整体结构框架。

第二章为技术介绍，主要介绍开发过程中所采用的核心技术，包括前端开发框架、后端开发框架、网络爬虫技术及个性化推荐算法的核心原理，为后续系统设计提供技术支撑。

第三章为概要设计，主要是根据用户的需求进行分析，划分平台的核心功能模块，确定系统的整体架构设计与关键业务流程。

第四章为数据库设计，主要介绍系统中核心实体，实体间联系以及数据库表的结构、索引及数据存储方案。

第五章为详细设计，主要针对系统中各功能模块展开具体设计，明确各模块的实现路径与技术细节。

第六章为系统实现，主要基于详细设计方案进行编码开发，重点说明核心功能的实现思路、关键代码片段，实现项目的最终落地。

第七章为系统功能演示，通过文字描述、界面截图形式，全面地展示平台的核心功能效果及操作流程。

2 相关技术

2.1 后端

2.1.1 Django Web 框架

Django^[3]是基于 Python 语言的开源 Web 开发框架，选择该框架的原因有三：

第一，Django 具有 ORM 机制，即对象关系映射机制。通过将数据库表结构映射为 Python 类，开发者无需编写原生 SQL 语句，即可通过类方法实现数据的增删改查操作。

第二，Django 具有 URL 路由系统支持灵活的请求分发机制，通过正则表达式匹配请求路径，将不同模块的请求定向至对应的视图函数，实现了业务逻辑的解耦。

第三，其自带的 Admin 后台管理系统，可通过简单配置实现车型数据、用户信息的可视化管理，方便管理员快速完成数据录入、更新与审核，降低平台运营维护成本。

2.1.2 协同过滤推荐算法

协同过滤技术是一种利用余弦相似度作为用户偏好相似性的量化指标，从而挖掘用户群体的行为模式来发现潜在的偏好关联的技术^[4]。

该技术将每个用户的评分记录视为多维空间中的一个向量，两个用户之间的相似程度则由其向量夹角的余弦值来反映。

2.1.3 网络爬虫技术

网络爬虫技术^[5]是实现该系统的数据获取的技术核心，通过采集主流汽

车平台的车型配置参数信息为推荐算法和汽车信息可视化等功能提供数据支撑。

Requests 库发送 http 请求，通过 lxml 的 XPath 解析 HTML 结构获取 JSON 格式数据，避免了页面渲染带来的效率损耗。

2.2 前端

2.2.1 Echarts

ECharts 是一款基于 JavaScript 的数据可视化库^[6]，支持折线图、柱状图、雷达图、散点图等多种图表类型，具有高兼容性、可定制性强、交互友好等核心优势，也是本次开发中数据可视化功能的核心实现工具。在选车推荐平台中，车型参数对比、市场数据展示等场景对数据可视化的需求较高，而 ECharts 能够将抽象的数值转化为直观的图表，有效降低用户理解成本。

2.2.2 Bootstrap

Bootstrap 是一款基于 HTML、CSS 和 JavaScript 的前端开发框架^[7]，提供了丰富的 UI 组件、响应式布局方案与交互插件，能够快速构建美观、兼容多设备的 Web 界面。本次开发选用 Bootstrap 5 版本，主要基于其轻量化、响应式设计与良好的兼容性，能够大幅降低前端开发工作量，同时保证界面风格的统一性。

3 概要设计

3.1 需求分析

本系统是为了给有购车需求的普通用户提供个性化选车推荐服务，同时也为系统管理员提供高效且方便的管理端支持。本系统通过搭建用户模块、车辆数据分析模块、个性化推荐模块、汽车信息对比模块等核心功能模块，精准满足用户与管理员这两类角色的需求。

3.1.1 管理员

(1) 用户信息管理，支持对注册用户信息进行新增、删除和查询操作。

(2) 车辆信息管理，负责维护系统内的车辆基础数据，包括新增车型信息（如品牌、型号、价格、配置、动力类型、油耗 / 电耗等核心参数）、修改已录入车辆的信息（如价格调整、配置更新）、删除下架车型数据，以及按品牌、价格区间、车型类别等维度查询车辆信息。

(3) 系统数据统计与监管，可查看系统的核心运行数据，包括用户注册量、汽车信息种类、汽车朋友圈监管、增删管理员账户等；同时监管用户行为，排查言语违规等行为，保障系统正常运行。

3.1.2 用户

(1) 个人信息与需求管理，用户完成注册登录后，可完善个人基础信息，如姓名、联系方式、地址，密码等，同时可随时修改已提交的选车需求，更新自身的购车偏好。

(2) 个性化推荐结果查看，系统根据海量用户对各类车型的打分，利用协同过滤推荐算法推荐给用户定制化车型推荐。用户可点击进入车型详情

页，查看完整的配置表、车型图片、用户口碑等信息。

(3) AI 客服辅助选车，用户在遇到选车困难时可随时咨询系统设置的智能客服获得及时有效的问题解答。

(4) 汽车朋友圈功能，用户可以在平台发布自己对某个汽车或品牌的看法，与其他用户交流互动，补齐信息差，优化选车准确度。

(5) 辅助选车功能，用户可使用系统提供的车型对比功能，自主选择多款车型进行参数、价格、配置的横向对比；也可根据价格区间、品牌、车型类别等条件手动筛选车型，补充个性化推荐结果的不足，进一步缩小选车范围。

3.2 功能模块设计

根据上述的需求分析，用户选车推荐平台将分为以下模块：

(1) 用户模块：提供基本的个人信息管理功能，支持修改账号、密码、头像、联系方式等基础资料，同时提供“汽车朋友圈”社区功能，用户可发布带文字、图片和视频的帖子，分享用车体验，支持评论、点赞和帖子搜索，打造车主交流的互动场景。

(2) 数据分析模块：提供可视化图表向用户展示汽车市场的宏观情况，包括市场销量、价格分析、能源分析、品牌分析四个维度的分析内容，帮助用户快速了解市场趋势与产品特点。

(3) 个性化推荐模块：基于用户浏览、评分等行为数据，通过协同过滤算法为用户推荐匹配其偏好的车型。

(4) 智能客服模块：为用户提供对话式的在线咨询服务，用户可通过自

然语言提问，快速获取车型参数、配置对比、选购建议等专业解答。

(5) 汽车对比与推荐模块：支持用户同时添加多款车型进行参数横向对比，通过表格和可视化图表直观呈现差异。同时提供场景化智能推荐和自定义权重推荐两种方式，用户可根据自身用车场景或偏好设置参数权重，系统自动计算并输出最优车型。

(6) 数据采集模块：从公开平台定向爬取汽车基础数据，包含基础采集、条件筛选采集和定时自动采集三种模式，采集的数据可直接清洗入库，并提供操作日志查看，保证数据的时效性和准确性。

(7) 系统管理模块：管理员可对平台进行全维度维护，包括用户信息管理、汽车数据管理、社区内容审核、权限分配等；同时支持对不同角色管理员进行权限配置，保障平台安全、有序运行。

本平台各模块及其详细子功能如系统功能结构图所示。

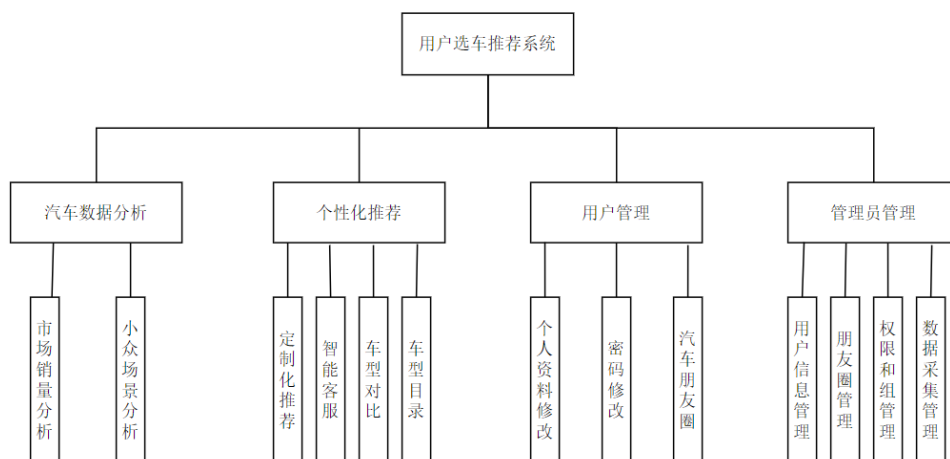


图 3-1 系统功能结构图

4 数据库设计

4.1 数据库简介

MySQL^[8] 是一个开源的、跨平台的 关系型数据库管理系统 (RDBMS)。它使用 SQL（结构化查询语言）来管理和操作数据，是目前世界上最流行的数据库之一。MySQL 的开源性、高性能、多引擎支持和易用性，使其成为个人开发者、中小企业和大型互联网公司首选的数据库管理系统。

其中,SQL 可分为: DQL(数据查询语句)、DDL(数据定义语句)、DML(数据操作语句)、TCL（事务控制语句）、DCL（数据控制语言）。

4.2 概念结构设计

4.2.1 E-R 图

- (1) 用户实体部分: 包含的属性有用户名、性别、头像、地址、个性签名、密码。
- (2) 汽车信息主体部分: 包含的属性有销量、尺寸、品牌、油耗、车型名称、图片、座椅配置、上市时间、厂商、空调控制、能源类型、价格、百公里加速。
- (3) 用户评价实体部分: 包含的属性有评分、评论内容、汽车 ID、用户 ID、发布时间。
- (4) 汽车朋友圈实体部分: 包含的属性有帖子内容、用户 ID、发布时间。
- (5) 朋友圈评论实体部分: 包含的属性有用户、发布时间、帖子、评论内容。

(6) 违禁词实体部分：包含的属性有违禁词、创建时间、更新时间。

(7) 管理员实体部分：包含的属性有用户名、密码、邮箱、姓名、最后登录时间、是否为超级管理员。

4.2.2 实体间联系图

实体联系图主要表现了不同实体之间的联系。如下：

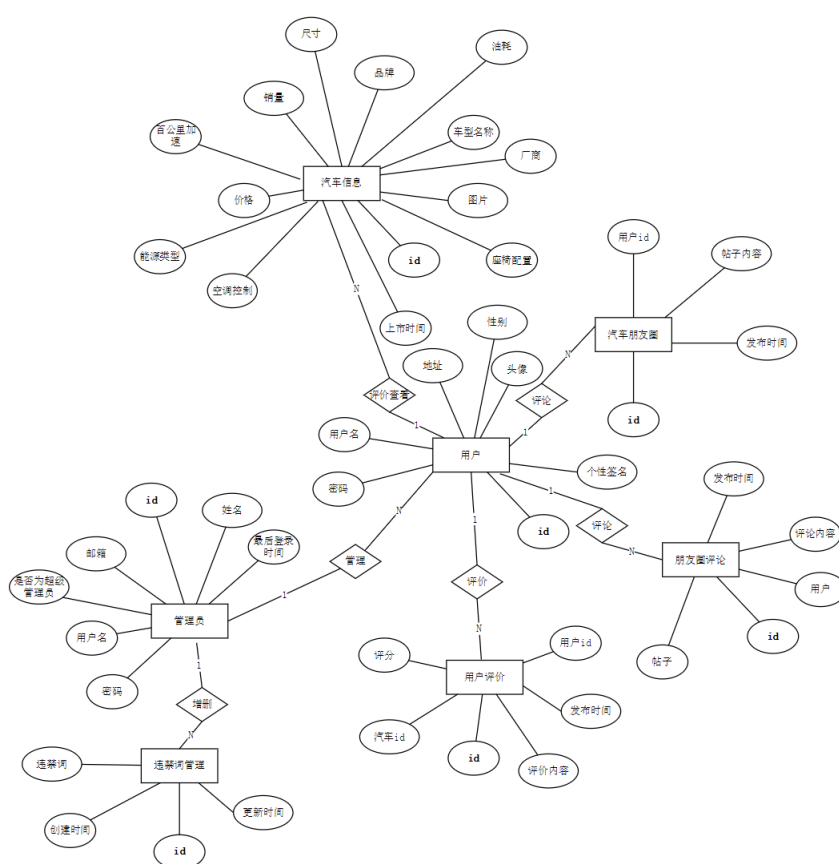


图 4-1 实体联系图

4.3 逻辑结构设计

根据本文对平台系统的需求分析，并得到的实体 E-R 图以及实体联系图，共设计了 7 张核心业务表，以下将这 7 个表详细展示数据库的具体设计：

(1) 用户信息表：用户信息表用于存储平台所有普通用户的基本信息，包括用户名、加密密码、个人资料如头像、性别、地址、个性签名，以及账号创建时间等。其中，用户 ID (id) 作为主键唯一标识一个用户。

表 4-1 用户表 User

字段名	字段类型	可否为空	备注
id	bigint	NOT NULL	主键
username	varchar(50)	NOT NULL	用户名
password	varchar(255)	NOT NULL	密码
avatar	varchar(255)	NULL	头像
gender	varchar(10)	NULL	性别
address	varchar(255)	NULL	地址
signature	varchar(255)	NULL	个性签名
created_at	datetime	NULL	创建时间

(2) 汽车信息表：汽车信息表用于存储汽车的核心参数与详细信息，包括品牌、车型名称、价格区间、车身尺寸、油耗、能源类型等。每辆汽车由唯一的 ID 标识，这些信息主要用于在前端展示汽车详情，供用户浏览和对比。

表 4-2 汽车信息表 CarInformation

字段名	字段类型	可否为空	备注
id	bigint	NOT NULL	主键
brand	varchar(50)	NULL	品牌
carname	varchar(100)	NULL	车型名称
carmodel	varchar(50)	NULL	车型

字段名	字段类型	可否为空	备注
carimg	varchar(255)	NULL	车辆图片
salevolume	int	NULL	销量
price	varchar(100)	NULL	价格区间
manufacturer	varchar(100)	NULL	制造商
dimensions	varchar(100)	NULL	车身尺寸
fuelconsumption	varchar(50)	NULL	油耗
marketime	varchar(50)	NULL	上市时间
insure	varchar(255)	NULL	保修信息
acceleration	varchar(50)	NULL	百公里加速
accontrol	varchar(50)	NULL	空调控制
energytype	varchar(50)	NULL	能源类型

(3) 用户评价表: 用户评价表用于记录用户对特定汽车的评价信息。每条评价记录包含评价内容、评分以及评价时间, 并通过汽车 ID 与用户 ID 分别关联到汽车信息表和用户信息表, 以实现特定用户对特定车型进行特定评论的查询功能。

表 4-3 用户评价表 CarComment

字段名	字段类型	可否为空	备注
id	bigint	NOT NULL	主键
car_id	bigint	NOT NULL	汽车 ID
user_id	bigint	NOT NULL	用户 ID
content	longtext	NULL	评价内容
score	int	NULL	评分 1-5

字段名	字段类型	可否为空	备注
create_time	datetime	NULL	评价时间

(4) 汽车朋友圈表: 汽车朋友圈表用于存储用户发布的动态内容。每条帖子记录发布用户的 ID、帖子正文内容以及发布时间, 供其他用户浏览和互动。

表 4-4 汽车朋友圈表 CarMoment

字段名	字段类型	可否为空	备注
id	bigint	NOT NULL	主键
user_id	bigint	NOT NULL	发布用户 ID
content	longtext	NULL	帖子内容
created_at	datetime	NULL	发布时间

(5) 朋友圈评论表: 朋友圈评论表用于存储用户对汽车朋友圈中帖子的评论信息。每条评论记录关联一个特定的帖子、评论者用户 ID、评论内容以及评论时间。通过该表, 每个帖子都可以拥有多条评论, 形成用户间的互动讨论。

表 4-5 朋友圈评论表 MomentComment

字段名	字段类型	可否为空	备注
id	bigint	NOT NULL	主键
moment_id	bigint	NOT NULL	帖子 ID
user_id	bigint	NOT NULL	评论用户 ID
content	longtext	NULL	评论内容
created_at	datetime	NULL	评论时间

(6) 违禁词表: 违禁词表用于存储内容安全相关的敏感词汇。表中每个违

禁词均有唯一 ID 标识，并记录创建和更新时间。平台在用户发布评价或朋友圈帖子时，可依据此表对内容进行过滤，屏蔽或提醒包含违禁词的文本，以维护社区环境。

表 4-6 违禁词表 BannedWord

字段名	字段类型	可否为空	备注
id	bigint	NOT NULL	主键
word	varchar(100)	NOT NULL	违禁词
created_at	datetime	NULL	创建时间
updated_at	datetime	NULL	更新时间

(7) 管理员信息表: 管理员信息表用于存储平台后台管理员的账号与权限信息。与普通用户表分开设计，增强了安全性。表中记录了管理员账号、密码、邮箱、姓名、最后登录时间以及是否为超级管理员的标识。超级管理员通常拥有系统的全部管理权限。

表 4-7 管理员信息表 app_adminuser

字段名	字段类型	可否为空	备注
id	bigint	NOT NULL	主键
username	varchar(50)	NOT NULL	管理员账号
password	varchar(255)	NOT NULL	密码
email	varchar(100)	NULL	邮箱
first_name	varchar(50)	NULL	姓名
last_login	datetime	NULL	最后登录时间
is_superuser	tinyint	NULL	是否超级管理员

5 详细设计

5.1 数据分析模块

5.1.1 市场销量分析设计

本模块以市场数据可视化分析^[9]为核心,将市场数据划分为四大分析方向,每个方向包含 4-5 张不同维度的图表,采用饼图、柱状图、折线图、散点图等多种形式呈现数据,实现对汽车市场的结构、趋势、分布的全方位展示,帮助用户快速理解市场特征与用户偏好。

(1) 市场概览模块:市场概览模块聚焦整体市场格局,为用户提供宏观层面的市场画像,提供五张图表,制造商市场份额表展示各大汽车制造商的市场占有率,品牌销量排行表展示销量最高的汽车品牌及其销量数据,销量 TOP10 车型详情表展示销量最高的 10 款车型及其详细数据,销量增长率趋势表展示月度销量同比增长率,反映市场增长趋势。月度销量趋势表展示月度销量趋势,反映市场季节性特点。

(2) 价格分析模块:价格分析模块聚焦市场价格维度的分布与规律,为用户提供基于价格的市场洞察,提供四张图表,价格区间分布表分析不同价格区间的车型数量分布。各品牌平均价格排行表展示各品牌车型的平均价格。各车型类别平均价格对比表展示不同车型类别的平均价格水平。价格与销量的关系散点图展示价格与销量的关系。

(3) 能源分析模块:能源分析模块聚焦不同能源类型车型的市场表现,反映新能源转型趋势,提供四张图表,能源类型销量分析图表展示不同能源类型汽车的销量数据。能源类型详细对比图表对比不同能源类型汽车的

多维度数据。能源效率对比图表对比不同能源类型的平均续航里程（电动车）和平均油耗（燃油车）。各能源类型平均功率对比表对比不同能源类型汽车的平均最大功率，反映动力性能差异。

（4）品牌分析模块：品牌分析模块聚焦各品牌的市场表现与竞争格局，为用户提供品牌维度的市场洞察，提供四张图表。品牌国别分布表展示国产、合资、进口品牌的市场分布。品牌车型数量分布表展示各品牌旗下的车型数量。车型分布表展示不同车型的市场分布比例。上市趋势表分析近年来各年度新车型的上市数量变化趋势。

用户进入市场销量模块后，系统会读取数据库数据，按照算法进行数据整合，生成图表供用户查看。

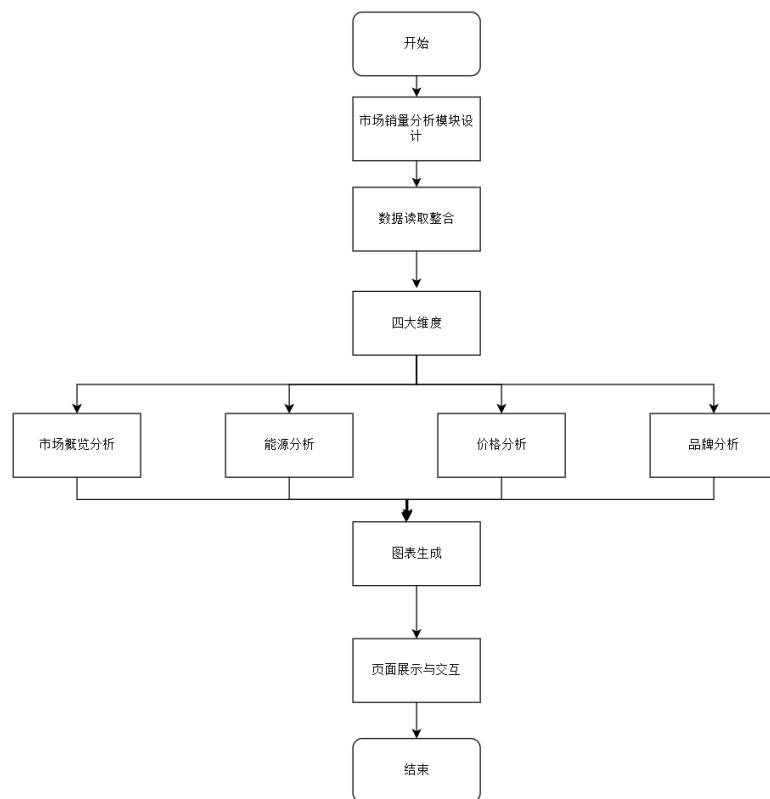


图 5-1 市场销量分析业务流程图

5.1.2 小众场景分析设计

小众场景分析模块共设计 5 张核心数据表，分别对应网约车、家庭用车、能源类型对比、商务用车、自驾游 5 大场景；

（1）网约车推荐数据表：存储各车型在网约车场景下的价格、能源类型、运营成本得分，用于推荐指数计算^[10]。

（2）家庭用车推荐数据表：存储各车型在家庭用车场景下的安全性、空间、舒适度、燃油经济性得分，用于综合推荐计算^[11]。

（3）能源类型对比数据表：存储不同能源车型的购车成本、使用成本、环保指数、续航便利性得分，用于能源类型推荐排名。

（4）商务用车推荐数据表：存储各车型在商务场景下的品牌形象、舒适度、性价比得分，用于商务车型推荐计算。

（5）自驾游推荐数据表：存储各车型在自驾游场景下的通过性、空间、可靠性、油耗得分，用于自驾车型推荐计算^[12]。

用户进入小众场景分析模块后，系统会展示默认权重下的五大图表，用户可根据自身偏好滑动滑块，系统进行权重归一化计算，计算出各车型综合得分之后刷新图表。

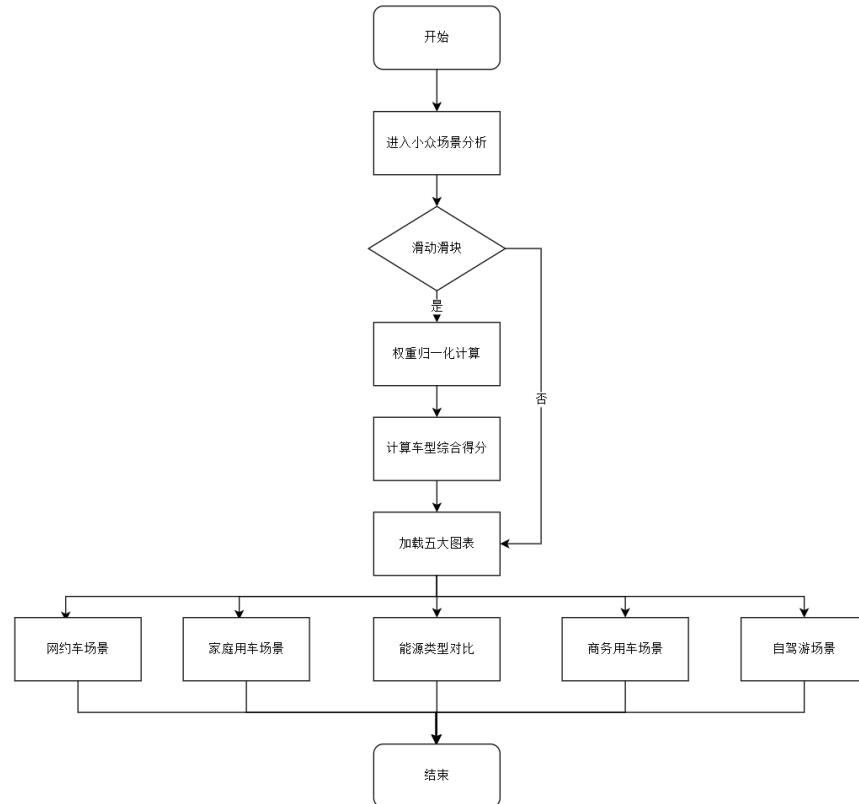


图 5-2 小众场景业务流程图

5.2 个性化推荐模块设计

个性化推荐模块核心基于用户协同过滤算法实现，通过计算用户相似度、推荐相似用户偏好的车型，达成精准个性化推荐。算法层处理数据稀疏问题（无评分时降级为销量推荐），交互层通过评分引导、结果标注增强个性化体验。

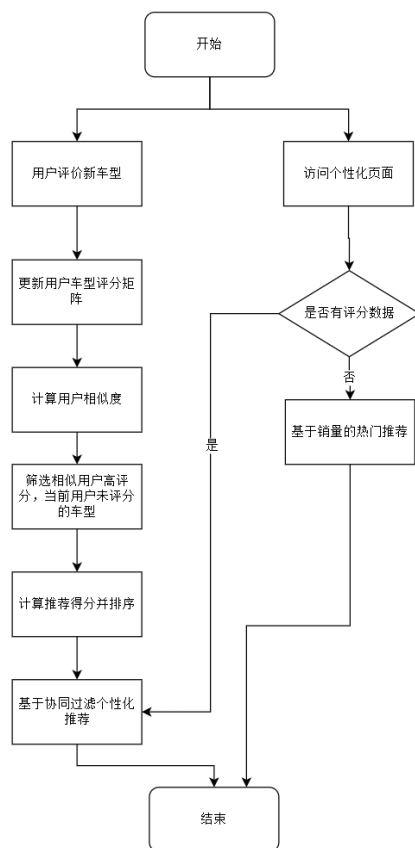


图 5-3 个性化推荐业务流程图

5.3 智能客服模块设计

智能客服模块采用聊天对话式交互，通过调用 Deepseek API 实现对话，是整个系统的智能服务入口，为用户提供即时购车咨询服务。

5.4 汽车对比模块设计

汽车对比与智能推荐模块是整个系统的核心决策功能，面向有购车需求的用户，提供集多车型参数对比，多场景智能推荐，自定义权重评分三位一体服务。本模块分为三大核心功能区：

（1）场景化智能推荐^[1]：用户可在网约车场景，家庭用车场景，能源类型对比，商务用车场景，自驾游场景获得系统提供的智能推荐。

(2) 自定义权重推荐：用户可自由调整价格、动力、舒适、空调、能源五大维度的权重，权重之和要求为 100%，系统按个人偏好打分推荐。

(3) 多车型可视化对比^[2]：支持最多 4 辆车同时对比，包含完整参数表格和六张 ECharts 专业图表，包括销量对比，价格对比，动力性能对比，舒适度评分，空调性能评分，综合性能雷达图，从而直观展示差异。

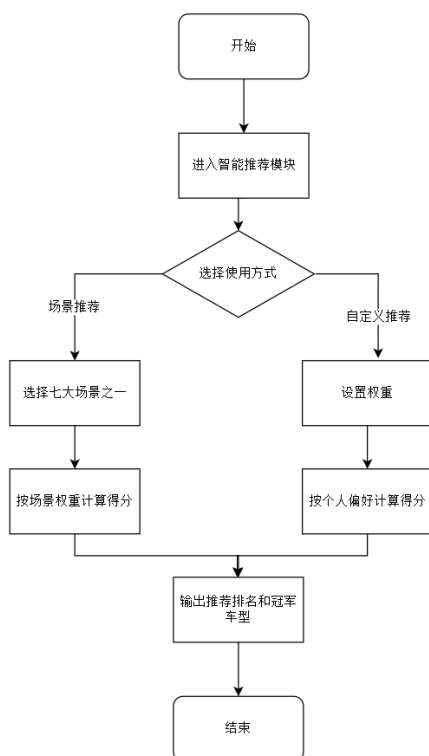


图 5-4 智能推荐业务流程图

5.5 用户模块设计

5.5.1 汽车朋友圈

该模块模拟社交朋友圈，可以实现用户帖子的发布，评论，让不同车主的心声被看到，同时支持搜索帖子功能，可根据帖子内容或发布者 ID 进行搜索，帖子可支持附带一份视频及最多九张照片。

5.5.2 资料修改

该模块可为用户提供修改个人 ID，密码，头像，地址，性别及个性签名等功能，同时提供发布帖子，查看个人主页，浏览朋友圈功能。

5.6 数据采集模块设计

通过爬取懂车帝汽车排行榜数据来获得汽车配置信息，提供三大功能，基础采集、条件筛选采集和定时自动采集，同时实现日志可视化监管。

用户进入爬虫管理界面后，首先选择采集方式，如果选择基础爬虫，先设置爬取数量，随后系统向汽车排行榜 API 发出 http 请求获取页面，随后用 xpath 解析 html 页面获取汽车配置数据，将数据保存为 csv 文件，进行数据清洗和去重之后存入数据库中，如果选择条件爬虫则会首先设置筛选条件，随后进入爬虫流程，系统会一一比对爬取数据是否符合筛选条件，如果采用定时爬虫，系统会首先创建定时任务存入内存字典中，随后线程会反复遍历任务列表，一旦到达时间后会自动执行基础爬虫。

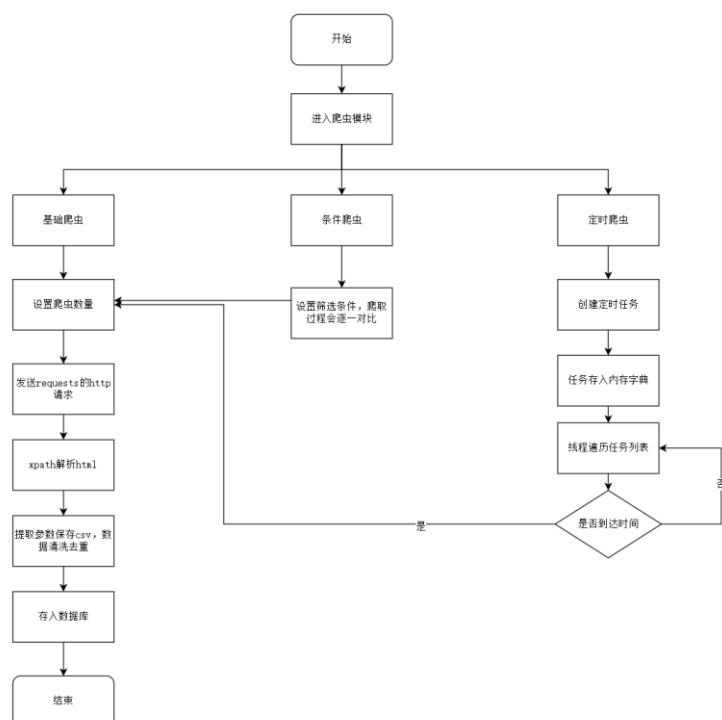


图 5-5 数据采集模块业务流程图

5.7 管理员模块设计

该模块可实现三大类管理，信息管理，社区管理和认证授权管理。

- (1) 信息管理：管理员可实现对前台用户，汽车信息及汽车评论的增删改。
- (2) 社区管理：管理员可实现对汽车朋友圈和违禁词的增删改。
- (3) 认证授权：最高权限管理员可对其他管理员的权限进行管理，设置权限组进行归类。

6 系统实现

6.1 市场销量分析实现

市场销量分析模块按照四大分析方向，即市场概览分析、价格分析、能源分析和品牌分析进行实现。每个方向包含多个可视化图表，后端通过 `CarDataAnalysis` 工具类提供数据接口，前端使用 `ECharts` 进行图表渲染。

6.1.1 市场概览模块实现

鉴于篇幅，销量 TOP 车型榜单、销量增长率实现和月度销量趋势实现模块的代码实现遵循相同的设计模式，下文仅以制造商份额分析和品牌销量排行实现模块为例进行详细说明。

(1) 制造商市场份额实现：后端实现时，首先使用 `aggregate` 方法配合 `Sum` 聚合函数计算数据库中所有汽车的总销量作为分母，然后按制造商字段分组，使用 `annotate` 配合 `Sum` 计算各制造商的总销量。获得各制造商的销量数据后，遍历结果集，计算每个制造商的市场份额百分比，最终返回制造商名称列表和对应的市场份额百分比列表。后端代码存于 `CarDataAnalysis.py` 中，具体如下：

```

@staticmethod
def get_manufacturer_market_share():
    """
    获取各制造商的市场份额
    """
    total_sales = CarInformation.objects.aggregate(total=Sum('salevolume'))['total']
    result = CarInformation.objects.values('manufacturer').annotate(
        total_sales=Sum('salevolume')
    ).order_by('-total_sales')

    # 计算市场份额百分比
    for item in result:
        item['market_share'] = round((item['total_sales'] / total_sales) * 100, 2)

    return {
        'manufacturers': [item['manufacturer'] for item in result],
        'market_shares': [item['market_share'] for item in result]
    }

```

图 6-1 制造商市场份额实现

(2) 品牌销量排行实现：后端实现时，首先使用 ORM 的 values 方法按品牌字段进行分组，然后通过 annotate 方法配合 Sum 聚合函数计算每个品牌下所有车型的销量总和，最后调用 order_by 方法按总销量降序排序，并使用切片操作[:limit]限制返回前 10 个品牌。返回格式为包含品牌名称列表和对应销量数值列表的字典。后端代码存于 CarDataAnalysis.py 中，具体如下：

```

@staticmethod
def get_top_brands_by_sales(limit=10):
    """
    获取销量最高的品牌
    """
    result = CarInformation.objects.values('brand').annotate(
        total_sales=Sum('salevolume')
    ).order_by('-total_sales')[:limit]

    return {
        'brands': [item['brand'] for item in result],
        'sales': [item['total_sales'] for item in result]
    }

```

图 6-2 品牌销量排行实现

6.1.2 价格分析模块实现

价格分析模块分为四大分析图表，分别是价格区间分布、价格与销量关联分析、品牌平均价格对比、车型平均价格对比图表。

鉴于篇幅，价格与销量关联分析、品牌平均价格对比和车型平均价格对比图表模块的代码实现遵循相同的设计模式，下文仅以价格区间分布实现模块为例进行详细说明。

(1) 价格区间分布实现：后端实现时，调用 `get_price_range_distribution` 方法，首先定义六个价格区间的计数器，遍历所有车型，对每条记录的价格字符串进行解析。由于数据库中价格字段存储格式为区间，方法包含专门的价格解析逻辑，解析得到价格数值后，取其中的最小值作为价格基准，根据最低价格落入的区间范围对对应的计数器加一。后端代码存于 `CarDataAnalysis.py` 中。

6.1.3 能源分析模块实现

能源分析模块包含能源类型销量分析、能源类型详细对比、能源效率对比和平均功率对比四个图表。

鉴于篇幅，能源类型详细对比和平均功率对比模块的代码实现遵循相同的设计模式，下文仅以能源类型销量占比实现和能源效率对比实现模块为例进行详细说明。

(1) 能源类型销量占比实现：后端实现时，首先通过 `values('energytype')` 函数按照能源类型字段对数据进行分组，将所有相同能源类型的车型归为一组。然后使用 `annotate` 方法配合 `Sum('salevolume')` 聚合函数计算每组内所

有车型的销量总和，最后调用 `order_by('-total_sales')` 函数按照总销量降序排序。后端代码存于 `CarDataAnalysis.py` 中。

```
@staticmethod
def get_sales_by_energy_type():
    """
    按能源类型统计销量
    """
    result = CarInformation.objects.values('energytype').annotate(
        total_sales=Sum('salevolume')
    ).order_by('-total_sales')

    return {
        'energy_types': [item['energytype'] for item in result],
        'sales': [item['total_sales'] for item in result]
    }
```

图 6-3 能源类型销量占比实现

(2) 能源效率对比实现：后端实现时，由于数据库中缺少现有的续航里程和油耗数据，因此采用基于行业调研数据的示例值。前端使用分组柱状图展示两种指标的对比，后端代码存于 `CarDataAnalysis.py` 中。

```
@staticmethod
def get_energy_efficiency():
    """获取能源效率数据"""
    try:
        # 这里需要根据实际数据计算续航和油耗
        # 暂时返回示例数据
        return {
            'energy_types': ['纯电动', '插电式混合动力', '汽油', '油电混合'],
            'ranges': [500, 150, 800, 900],
            'fuel_consumption': [0, 2.5, 7.5, 5.0]
        }
    
```

图 6-4 能源效率对比实现

6.1.4 品牌分析模块实现

品牌分析模块包含品牌国别分布、品牌车型数量分布、车型分布和上

市趋势四个图表。

鉴于篇幅，品牌车型数量分布和上市趋势模块的代码实现遵循相同的设计模式，下文仅以品牌国别分布和车型分布模块为例进行详细说明。

(1) 品牌国别分布实现：后端实现时，首先定义一个品牌国别映射字典，将常见品牌和其所属类别建立关联,然后遍历数据库中所有不重复的品牌，根据映射表确定每个品牌的国别类别分别统计各类别的品牌数量和销量总和。后端代码存于 CarDataAnalysis.py 的 get_brand_country_data 方法中。

(2) 车型分布实现：后端实现时，使用 values('carmodel')按车型类别分组，通过 annotate(count=Count('id'))统计每种类别下的车型数量，按数量降序排序后返回。后端代码存于 CarDataAnalysis.py 的 get_car_model_distribution 方法中，核心代码如下：

```
@staticmethod
def get_car_model_distribution():
    """
    获取不同车型的分布
    """
    result = CarInformation.objects.values('carmodel').annotate(
        count=Count('id')
    ).order_by('-count')

    return {
        'car_models': [item['carmodel'] for item in result],
        'counts': [item['count'] for item in result]
    }
```

图 6-5 车型分布实现

6.2 小众场景分析实现

小众场景分析模块针对不同用户群体的特定需求进行汽车推荐和分

析，包括网约车、家庭用车、能源类型对比、商务用车和自驾游在内的五大场景。该模块采用后端预处理加前端加权计算的混合架构，后端负责数据筛选、价格解析和各维度原始评分计算，前端负责权重调节和综合得分实时计算。

鉴于篇幅，能源类型对比、商务用车和自驾游模块的代码实现遵循相同的设计模式，下文仅以网约车和家庭用车为例进行详细说明。

6.2.1 后端辅助函数设计

在实现各场景分析之前，首先需要定义两个核心辅助函数，用于价格解析和得分计算。

(1) 价格区间提取函数：该函数用于提取价格区间的最小值和最大值。

```
def extract_price_range(price_str):
    """从价格字符串中提取最低价和最高价"""
    try:
        if not price_str or price_str == '无' or price_str == '未知':
            return None, None
        numbers = re.findall(r'\d+\.\d*', str(price_str))
        if numbers:
            if len(numbers) >= 2:
                return float(numbers[0]), float(numbers[1])
            elif len(numbers) == 1:
                return float(numbers[0]), float(numbers[0])
        return None, None
    except:
        return None, None
```

图 6-6 价格区间提取函数实现逻辑

(2) 归一化得分计算函数：该函数将原始数值转换为得分，支持正向和反向两种评分模式，并通过敏感度参数调节得分曲线的陡峭程度。

```
def calculate_score(value, min_val, max_val, reverse=False, sensitivity=1.5):  
    """  
    将值归一化到0-100分，增加敏感度  
    """  
    if value is None:  
        return 50  
    if max_val == min_val:  
        return 50  
  
    if reverse:  
        score = 100 - ((value - min_val) / (max_val - min_val) * 100)  
    else:  
        score = ((value - min_val) / (max_val - min_val) * 100)  
  
    normalized = score / 100  
    transformed = math.pow(normalized, 1.0 / sensitivity) if sensitivity > 1 else normalized  
    enhanced_score = transformed * 100  
  
    return max(0, min(100, enhanced_score))
```

图 6-7 归一化得分计算函数实现逻辑

6.2.2 网约车场景实现

网约车场景的算法包含数据筛选、价格归一化、能源得分和运营成本得分四个步骤。

(1) 价格筛选：系统遍历所有车型，调用 `extract_price_range` 函数提取价格区间的最低价格，只保留最低价低于 15 万元的车型。若符合条件的车型不足 10 辆，系统自动将筛选条件放宽至 20 万元，确保推荐列表有足够的支撑。

(2) 价格得分计算：价格得分的计算公式采用反向归一化，价格越低得分越高。

(3) 能源类型得分：调用 `get_energy_score` 函数获取能源类型得分，纯电动 95 分最高，插电混动 90 分次之，汽油 65 分较低。

(4) 运营成本得分：运营成本得分根据能源类型和油耗综合计算。纯电动直接给 95 分，混动给 90 分，汽油车型则根据百公里油耗计算：油耗 6L/100km 时得 80 分，每增加 1L 扣 3 分，最低不低于 40 分。

6.2.3 家庭用车场景实现

家庭用车场景从安全性、空间、舒适度和燃油经济性四个维度对车型进行评分。

（1）安全性得分：基于品牌安全口碑的预定义映射表。沃尔沃以 98 分位居首位，奔驰、宝马、奥迪等豪华品牌得分在 92-95 分之间，丰田、本田等合资品牌得分在 86-91 分之间。

（2）空间得分：空间得分通过车型名称关键词匹配实现。MPV 或商务车型得 95 分最高，SUV 或越野车型得 88 分，轿车得 75 分。

（3）舒适度得分：舒适度得分根据品牌档次判断。奔驰、宝马、奥迪、雷克萨斯等豪华品牌得 95 分，丰田、本田、日产、大众等主流合资品牌得 85 分，其他品牌默认 75 分。

（4）燃油经济性得分：燃油经济性得分复用 `get_energy_score` 函数，纯电动 95 分最高，混动 90 分次之，汽油 65 分。

6.3 个性化推荐实现

个性化推荐模块基于协同过滤算法构建，核心目标是根据用户的汽车评分行为，为用户推送符合其偏好的车型，解决“信息过载”问题，提升用户选车效率。代码实现位于 `app/recommendation.py` 文件中。

6.3.1 用户评分向量构建

对于有评分记录的用户，系统需要将用户的评分数据转换为向量形式，以便进行相似度计算。首先获取所有汽车 ID 的集合，建立汽车 ID 到向量索引的映射关系。然后遍历目标用户的评分记录，构建目标用户的评分向

量，向量的长度等于汽车总数，未评分的汽车位置填充为 0。

```
# 将目标用户的评分转化为向量
target_user_vector = np.zeros(len(all_car_ids))
car_id_to_index = {car_id: idx for idx, car_id in enumerate(all_car_ids)}

for car_id, rating in target_user_ratings.items():
    if car_id in car_id_to_index:
        target_user_vector[car_id_to_index[car_id]] = rating
```

图 6-8 用户评分转换向量

6.3.2 用户相似度计算

系统遍历除目标用户外的所有其他用户，为每个用户构建评分向量，然后使用 `sklearn.metrics.pairwise` 模块中的 `cosine_similarity` 函数计算两个向量的余弦相似度。

```
# 计算目标用户与其他用户之间的相似度得分
for other_user_id, other_ratings in user_ratings.items():
    if other_user_id == user_id:
        continue

    # 将其他用户的评分转化为向量
    other_user_vector = np.zeros(len(all_car_ids))
    for car_id, rating in other_ratings.items():
        if car_id in car_id_to_index:
            other_user_vector[car_id_to_index[car_id]] = rating

    # 计算余弦相似度
    similarity = cosine_similarity([target_user_vector], [other_user_vector])[0][0]
    user_similarity_scores[other_user_id] = similarity
```

图 6-9 计算余弦相似度

6.3.3 推荐结果生成

计算出所有用户的相似度后，系统按相似度得分降序排序，筛选出 Top-K 个相似用户，K 默认为 10。

6.4 智能客服实现

智能客服模块采用前后端分离的架构设计。前端提供聊天窗口界面，用户通过文本输入问题并查看回复。后端封装 Deepseek 大模型 API 调用逻辑

辑，处理用户请求并返回智能回复。模块通过环境变量配置 API 密钥和模型参数，支持灵活的部署配置。

`ask_doubao` 函数是智能客服模块的核心，负责向 Deepseek 大模型 API 发送请求并获取回复。该函数接收用户消息和对话历史作为参数，构造符合 API 要求的请求体，发送 POST 请求后解析响应内容。

```
def ask_doubao(message, history=None):
    if not is_configured():
        return {"error": True, "response": "API 未配置, 请检查.env 文件"}

    url = f"{DOUBAO_BASE_URL.rstrip('/')}/chat/completions"

    headers = {
        "Content-Type": "application/json",
        "Authorization": f"Bearer {DOUBAO_API_KEY}"
    }

    payload = {
        "model": DOUBAO_MODEL,
        "messages": history + [{"role": "user", "content": message}] if history else [{"role": "user", "content": message}]
    }

    try:
        response = requests.post(url, headers=headers, json=payload, timeout=50)
        response.raise_for_status()
        data = response.json()
        return {"error": False, "response": data["choices"][0]["message"]["content"]}
    except Exception as e:
        return {"error": True, "response": f"API请求失败: {str(e)}"}
```

图 6-10 对话请求核心函数

6.5 汽车对比实现

汽车对比与智能推荐模块是系统的核心决策功能，面向有购车需求的用户，提供多车型参数对比、多场景智能推荐和自定义权重推荐三位一体的服务。该模块将原本零散、复杂的汽车参数，转化为直观、可视化、量化的对比结果，帮助用户快速做出购车决策。

6.5.1 评分机制

价格评分采用反向归一算法，而动力评分、舒适度评分、空调评分和能源类型评分则采用阶梯式累加算法，具体累加算法如下：

（1）动力评分：根据是否具有最大功率、百公里加速阶梯式累加，分

别可得 50/75/85 分。

（2）舒适度评分：根据是否具有座椅功能、座椅材质阶梯式累加，分别可得 40/70/85 分

（3）空调评分：根据是否具有空调控制方式、后排空调、温度分区累加制 40/60/80/100 分。

（4）能源类型评分：根据车辆的能源类型直接映射到对应的分数值。汽油或柴油车型得 90 分，插电式混合动力得 70 分，纯电动得 30 分，其他类型默认 50 分。

6.5.2 推荐结果生成

通过参数表格和 ECharts 图表两种形式展示车型差异，帮助用户直观地进行横向比较。

（1）参数表格对比：将多款车型的配置参数并排展示，每一行对应一个参数项，每一列对应一款车型。为了帮助用户快速识别优劣，系统实现了最佳值自动高亮功能。

（2）图表对比：采用六种 ECharts 图表全面展示对比维度。柱状图用于展示单一维度的数值对比，雷达图用于展示多维度综合性能。

6.6 用户模块实现

6.6.1 个人信息修改

用户可以修改个人姓名，地址，性别，个性签名，密码，头像，用户修改后通过 POST 请求提交，后端接收数据并更新数据库中的对应用户记录。其中头像修改是后端接收后保存到服务器指定目录，数据库存储文件

路径而非文件本身。密码修改会设置原始密码检查，具体代码如下：

```
# 检查原始密码是否正确
if oldPwd != userInfo.password:
    return '原始密码不正确'

# 检查两次输入的新密码是否一致
if newPwd != newPwdConfirm:
    return '两次新密码不一致'

# 检查用户是否存在
try:
    user = User.objects.get(username=userInfo.username)
except User.DoesNotExist:
    return '用户不存在'

# 更新密码
user.password = newPwdConfirm
user.save()
```

图 6-11 修改密码逻辑

6.6.2 汽车朋友圈

在汽车朋友圈中，系统为用户提供信息流展示，搜索检索，点赞评论互动，贴子发布的功能，具体实现如下：

(1) 帖子发布流程采用表单提交方式。用户在发布页面输入文本内容、选择图片或视频后，点击发布按钮，前端将数据封装为表单发送至后端。后端接收后创建帖子记录，保存多媒体文件，并将文件路径关联到帖子记录。

(2) 信息流展示流程采用分页加载机制。用户访问朋友圈页面时，后端从数据库查询所有帖子，按创建时间倒序排列，每页显示固定数量的帖子。

(3) 搜索检索功能基于关键词模糊匹配。用户在搜索框输入关键词后，

系统在帖子的文本内容和发布者的用户名两个字段中进行模糊匹配，返回包含关键词的帖子列表。

(4) 点赞评论互动采用 Ajax 异步请求实现无刷新交互。用户点击点赞或者评论按钮时，前端发送 POST 请求到后端，后端判断该用户是否已经点赞过或评论过该帖子按照情况创建点赞或评论记录。

6.7 数据采集实现

该模块分为基础采集，条件筛选采集和定时采集三大块功能。

(1) 基础采集：爬虫使用 Python 的 requests 库向目标网站的 API 接口发送 HTTP 请求，通过设置 User-Agent 请求头模拟浏览器获取 JSON 格式的响应数据，解析后提取所需字段，对于更详细的配置参数，爬虫使用 lxml 库的 XPath 表达式进一步访问车型详情页面，从 HTML 中提取结构化数据。采集到的数据首先保存到 CSV 文件中作为原始备份，然后通过数据清洗去除空值和重复值，最后批量导入 MySQL 数据库。CSV 文件以时间戳命名，便于追溯每次采集的历史记录。

(2) 条件筛选采集：是在基础采集之上增加了数据过滤机制，允许管理员在启动爬虫前指定筛选条件，爬虫在获取每条数据后，会提取对应的字段值，与筛选条件进行比对，爬虫只采集符合条件的数据，跳过不符合的数据。

```
def check_filters(self, car_brand, car_model_type, car_energy):
    """
    检查汽车是否符合筛选条件
    :return: (是否匹配, 匹配原因列表)
    """
    if not self.filters:
        return True, []

    match = True
    match_reasons = []

    # 品牌筛选 (先精确匹配, 再模糊匹配)
    if '品牌' in self.filters and self.filters['品牌']:
        filter_brand = self.filters['品牌']
        if filter_brand == car_brand:
            match_reasons.append(f"品牌精确匹配: {filter_brand}")
        elif filter_brand.lower() in car_brand.lower():
            match_reasons.append(f"品牌模糊匹配: {filter_brand} -> {car_brand}")
        else:
            match = False
            match_reasons.append(f"品牌不匹配: 需要{filter_brand}, 实际{car_brand}")

    # 车型筛选 (先精确匹配, 再模糊匹配)
    if match and '车型' in self.filters and self.filters['车型']:
        filter_model = self.filters['车型']
        if filter_model == car_model_type:
            match_reasons.append(f"车型精确匹配: {filter_model}")
        elif filter_model in car_model_type:
            match_reasons.append(f"车型模糊匹配: {filter_model} -> {car_model_type}")
        else:
            match = False
            match_reasons.append(f"车型不匹配: 需要{filter_model}, 实际{car_model_type}")
```

图 6-12 条件筛选实现逻辑

(3) 定时采集：采用内存字典配合独立线程的轻量级调度方案。系统启动时创建一个后台线程，该线程每分钟遍历一次内存中的任务字典，内存中存储所有定时任务的完整配置，包括名称、时间、频率、爬取数量、筛选条件等，检查是否有任务到达执行时间。若匹配成功，则启动子进程执行爬虫脚本。

6.8 管理员模块实现

管理员模块基于 Django 框架自带的 Admin 后台进行二次开发，该模块通过注册数据模型到 Admin 站点，自动生成对应的列表页、详情页、编辑页和删除页，无需手动编写 HTML 代码，该系统在此基础上进行了定制化扩展。

6.8.1 信息管理实现

(1) 用户管理：通过 `UserAdmin` 类实现，在列表页展示用户 ID、用户名、性别、地址和注册时间等信息。

(2) 汽车信息管理：通过 `CarInformationAdmin` 类实现。列表页展示了 21 个字段，涵盖品牌、车型、销量、价格、制造商、车型类别、能源类型、上市时间、保修信息、性能参数、座椅配置、空调配置等全部数据维度。管理员可以按能源类型、车型类别、品牌进行筛选，也可以按品牌或车型名称进行搜索，并通过自定义 actions 实现多种批量操作功能：批量删除、批量标记为新能源汽车、批量清除销量数据。

(3) 汽车评论管理：通过 `CarCommentAdmin` 类实现，列表页展示评论 ID、评论用户、关联车型、评分、评论内容预览和评论时间。管理员可以按评分和时间筛选评论，也可以按用户名或车型名称搜索。

6.8.2 社区管理实现

(1) 朋友圈管理：通过 `CarMomentAdmin` 类实现，采用内联管理的方式整合帖子、图片、视频和评论的关联管理。管理员在编辑一条帖子时可以同时查看和管理该帖子的所有图片、视频和评论，无需跳转多个页面。

(2) 违禁词管理：通过 `BannedWordAdmin` 类实现，管理员可以添加、编辑、删除违禁词。

6.8.3 认证授权实现

认证授权模块基于 Django 内置的权限系统，通过 `django.contrib.auth` 应用管理管理员用户和权限组。

7 系统功能演示

7.1 用户模块

7.1.1 登录与注册

(1) 用户登录界面，用户输入用户名和密码即可登录，若无账号则需注册，点击创建账号，输入信息即可成功注册账号，若为管理员身份，点击右上角进入管理员登录界面，输入账号密码进入系统。



图 7-1 登录（左）注册（中）管理员（右）

7.1.2 资料修改

(1) 个人信息修改界面：用户进入该界面，可以对自己的用户名，地址，性别，个性简介和头像进行修改。同时可以在我的社交里点击发布分享到汽车朋友圈中发布帖子，点击我的分享即可查看自己发布过的帖子合集，点击浏览朋友圈即可一键跳转汽车朋友圈界面。

我的
个人信息 / 资料编辑页面

修改个人信息

用户名: 李四

地址: 广东省广州市番禺区华南理工大学

性别: 男

个人简介: 明天的事，明天再想

头像上传:  [选择文件](#) 未选择文件

[提交](#)

我的社交

[浏览朋友圈](#) [我的分享](#) [发布分享](#)

图 7-2 个人信息修改界面

(2) 修改密码界面：用户进入修改密码界面需要首先正确输入原密码，其次要对新密码进行二次确认。

密码重置
个人信息 / 修改密码页面

修改密码

原始密码: 请输入原始密码

输入新密码: 请输入新密码

新密码确认: 请再次输入新密码

[提交](#)

图 7-3 修改密码界面

7.1.3 汽车朋友圈

(1) 主界面：用户可浏览其他用户发布的帖子，了解具体车型的车主真实用车体验，并且可以点赞评论，形成用户互动交流的氛围。

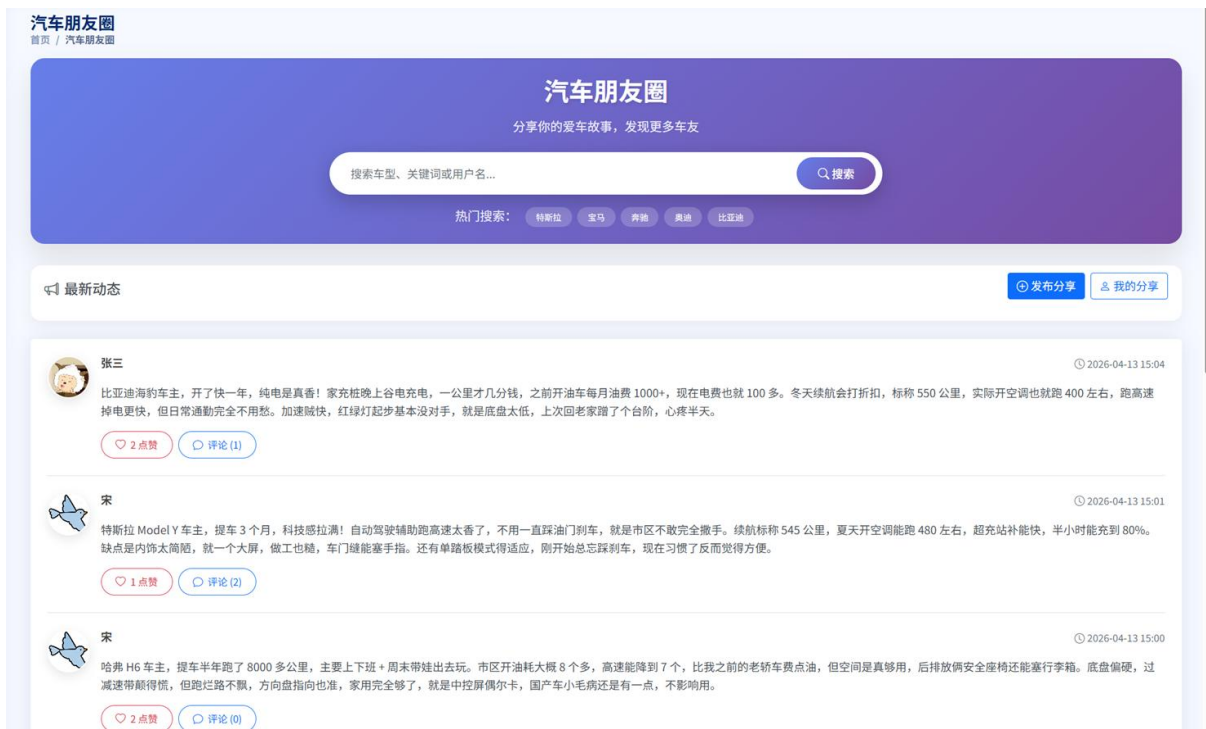


图 7-4 朋友圈主界面

(2) 可根据用户名或关键字搜索特定帖子，以搜索特斯拉为例，系统会筛选出汽车朋友圈中所有与特斯拉相关的帖子。



图 7-5 朋友圈搜索界面

7.2 数据分析

7.2.1 市场销量分析

市场销量分析分为四大类别，市场概览，价格分析，能源分析，品牌分析。

市场概览类有六张图表供用户查看，鉴于篇幅，下文仅以制造商市场份额和品牌销量排行图表为例进行详细展示：

（1）市场概览-制造商市场份额：用户进入制造商市场份额界面，可以根据自己的偏好勾选想要了解的汽车制造商品牌，点击应用筛选即可查看各品牌的市场占有份额。

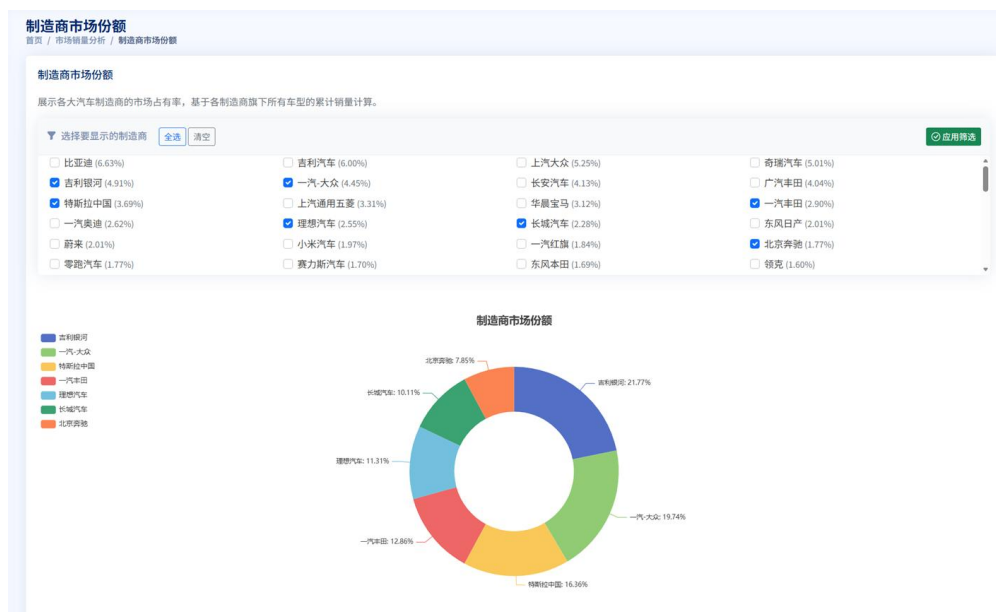


图 7-6 制造商市场份额表

（2）市场概览-品牌销量排行：用户进入品牌销量排行界面，可以根据自己偏好最多勾选六个汽车品牌，点击应用可查看各品牌销量总和以及排名。

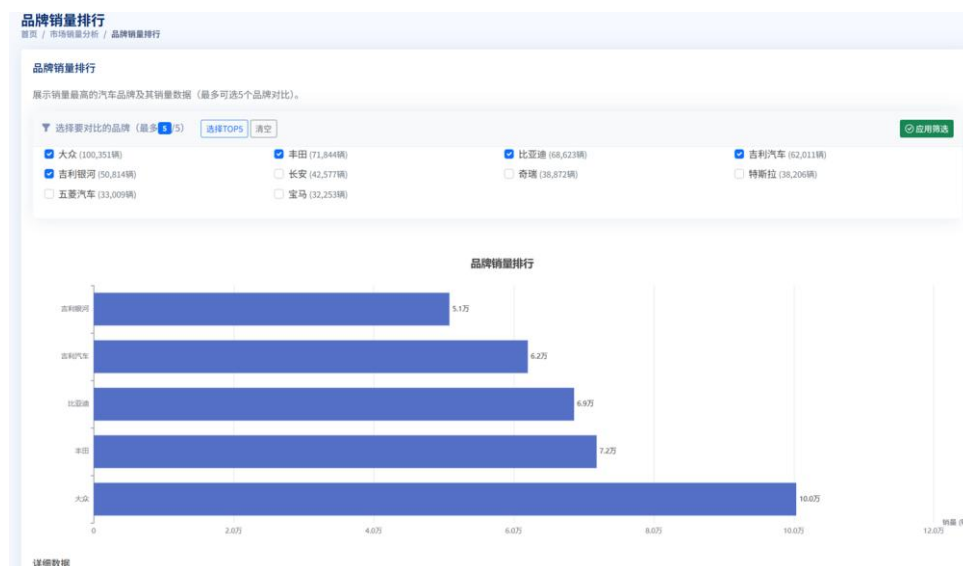


图 7-7 品牌销量排行表

价格分析类分为四张图表，鉴于篇幅，下文仅以各品牌平均价格排行和价格与销量的关系图表为例进行详细展示：

(1)价格分析-各品牌平均价格排行:用户进入各品牌平均价格排行界面,可以根据自己偏好最多勾选八个汽车品牌, 点击应用筛选可查看个品牌的所有车型平均价格和排名。



图 7-8 各品牌平均价格排行表

(2) 价格分析-价格与销量的关系: 用户进入价格与销量关系界面, 可以

根据自己偏好选择想了解的能源类型，即可生成散点图，其中 X 轴为价格，Y 轴为销量，可以通过观察点的分布来分析价格对销量的影响。

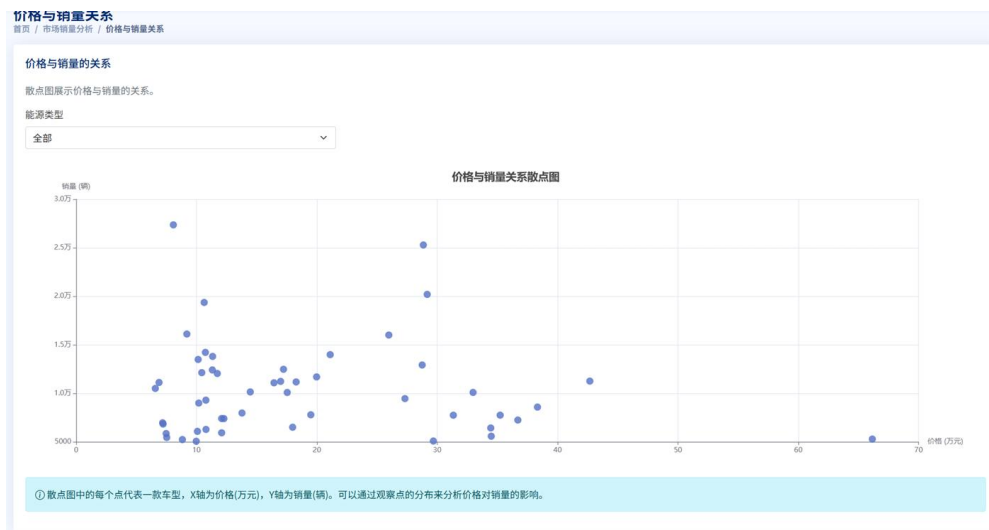


图 7-9 价格与销量的关系表

能源分析类分为四张表，鉴于篇幅，下文仅以能源类型销量分析和能源类型详细对比图表为例进行详细展示：

(1) 能源分析-能源类型销量分析：用户进入能源类型销量界面后可查看各能源类型的销量统计和排名。

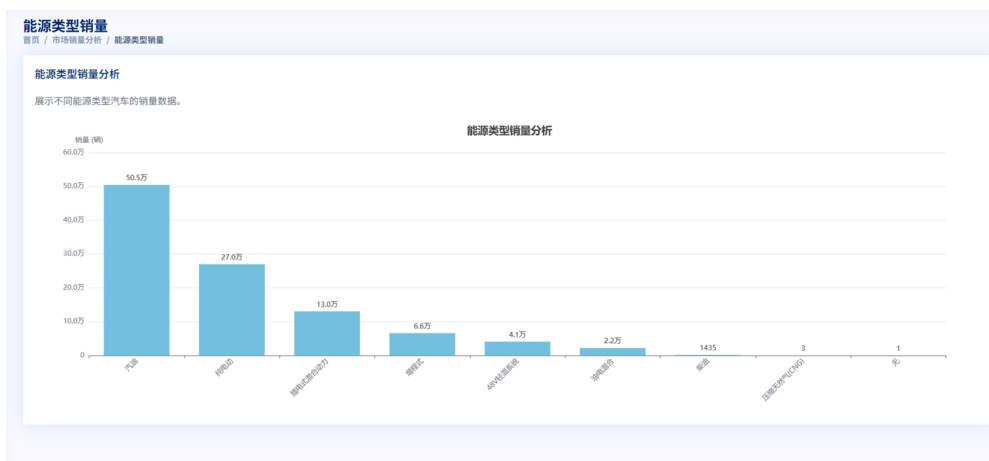


图 7-10 能源类型销量分析表

(2) 能源分析-能源类型详细对比：用户进入能源类型详细对比界面，可

可以根据自己偏好选择对比指标，品牌和能源类型，并根据图表类型生成相应的图表，便于用户了解一个品牌详细数据对比。



图 7-11 能源类型详细对比表

品牌分析类分为四张表，鉴于篇幅，下文仅以品牌国别分布和车型分布图表为例进行详细展示：

（1）品牌分析- 品牌国别分布：用户可以根据分析类型，图表类型以及品牌类型了解品牌的国别分布情况。

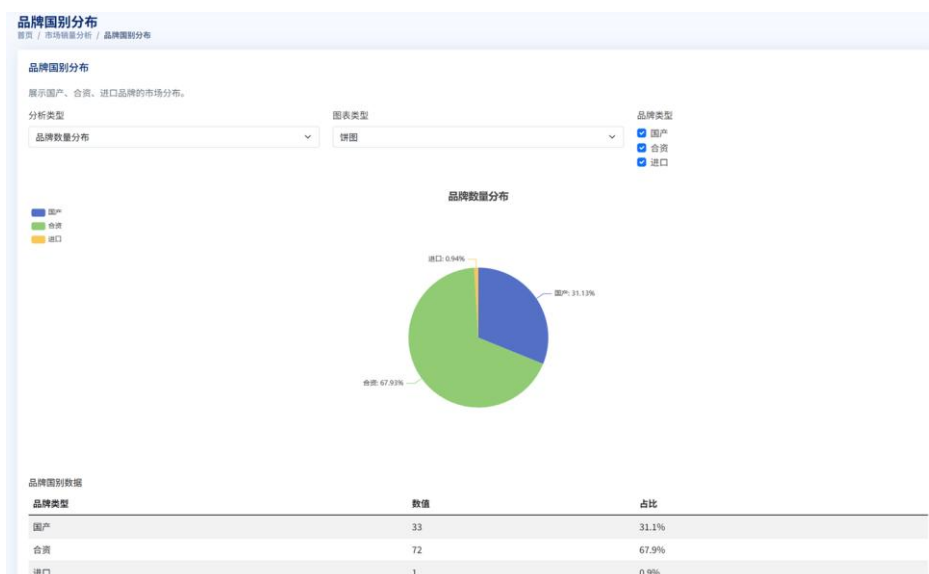


图 7-12 品牌国别分布表

(2) 品牌分析-车型分布：用户进入车型分布界面，可根据自己的偏好勾选最多八个车型，点击应用筛选即可查看不同车型的市场分布比例。

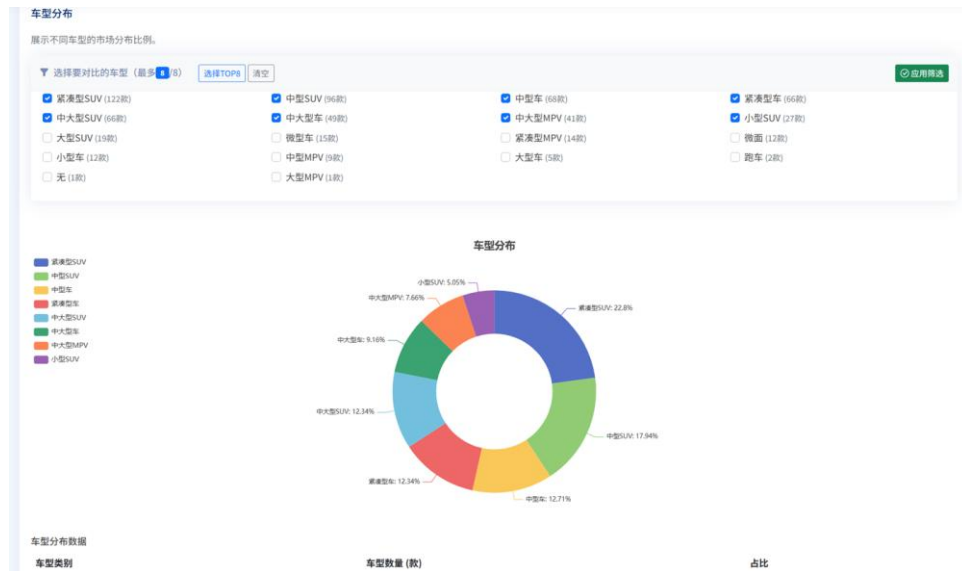


图 7- 13 车型分布表

7.2.2 小众场景分析

小众场景分析为用户提供多种场景的选车推荐，包括网约车推荐，家庭用车推荐，能源类型对比，商务用车推荐，自驾游推荐，鉴于篇幅，下文仅以网约车和家庭用车图表为例进行详细展示：

(1) 网约车推荐：用户进入网约车推荐排行榜，默认的权重是价格权重 30%，能源类型权重 40%，运营成本权重 30%，用户可滑动滑块调整图表。



图 7-14 网约车推荐表

(2) 家庭用车推荐：用户进入家庭用车模块，默认权重是安全性 35%，空间 35%，舒适度 20%，燃油经济性 10%，用户滑动滑块调整权重可更新图表。

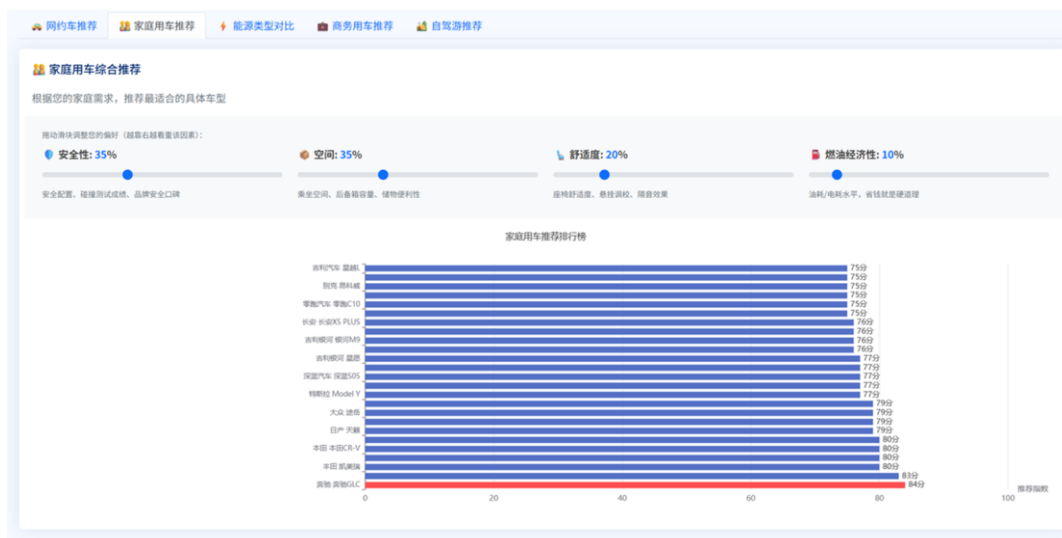


图 7-15 家庭用车推荐表

7.3 个性化推荐

在个性化推荐栏中，系统存留用户的历史打分与评价记录，结合协同过滤算法实现个性化推荐。

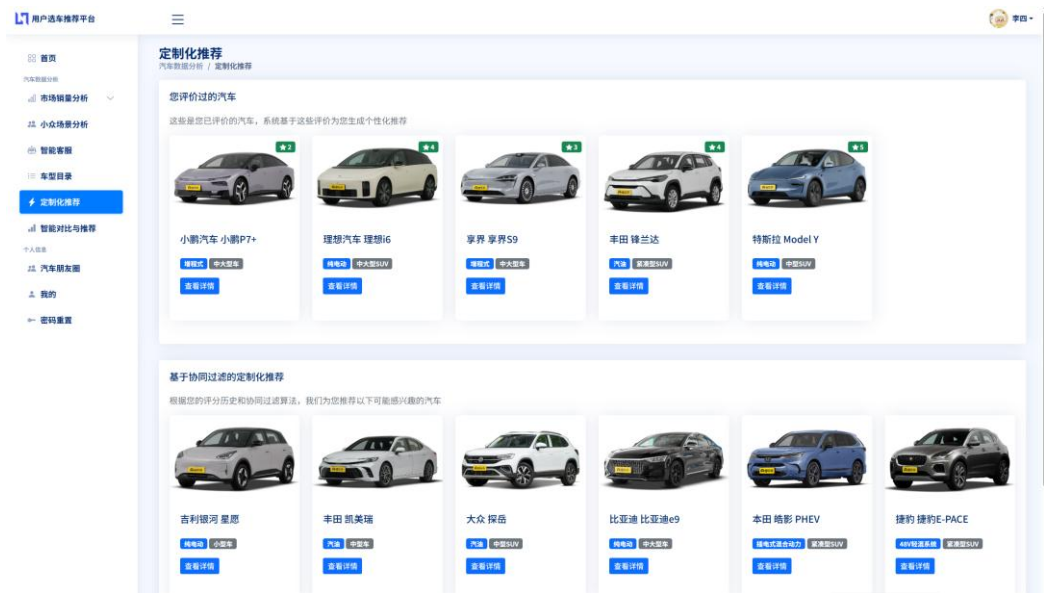


图 7-16 个性化推荐界面

7.4 智能客服

在智能客服栏中，用户可与聊天机器人进行对话，咨询有关汽车方面的困惑。

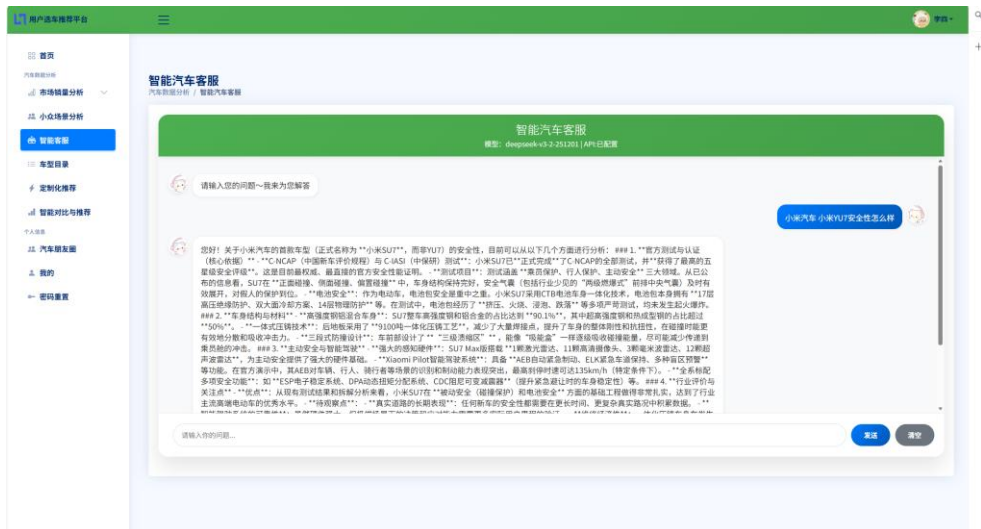


图 7-17 智能客服聊天界面

7.5 汽车对比

在车型目录中，首先可以看到一条跳转哔哩哔哩官网的弹窗，用户想了解更多汽车讲解视频即可一键跳转。

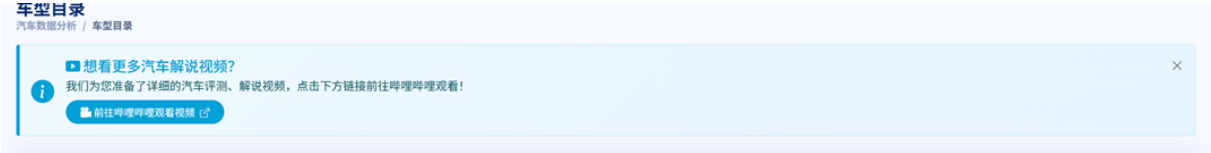


图 7-18 跳转哔哩哔哩界面

其次，本栏目提供用户偏好推荐，用户可根据自己的偏好滑动滑块，点击推荐一键最佳车型推荐。



图 7-19 用户偏好推荐

另外，用户可根据品牌，能源类型，车型甚至汽车名进行汽车搜索，亦可点击卡片直接进入汽车详情页查看相关信息。

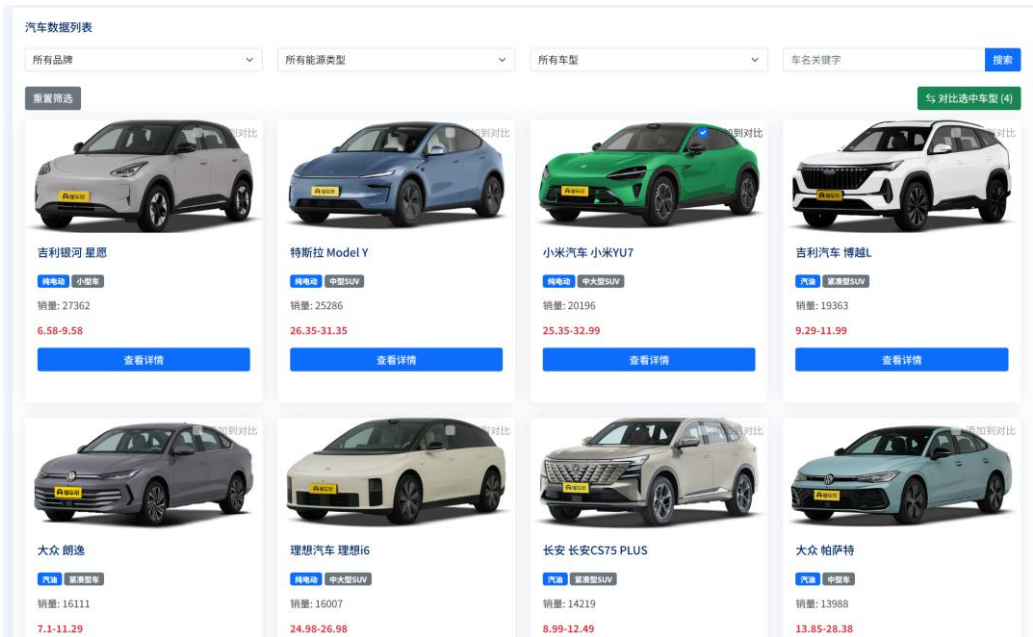


图 7- 20 车型目录表

进入汽车详情页可详细浏览汽车配置。

基本信息

品牌	小米汽车
车型名称	小米YU7
制造商	小米汽车
销量	20196 辆
价格区间	25.35-32.99
车型	中大型SUV
能源类型	纯电动
上市时间	2025.06
保修信息	无

性能参数

最大功率	235(320Ps)
车身尺寸	4999x1996x1608
百公里加速	5.88
综合油耗	无

座椅配置

座椅材质	无
前排座椅功能	加热、通风、记忆、按摩
第二排座椅功能	无

空调配置

空调控制方式	自动
后排独立空调	标配
后排出风口	标配
温度分区控制	三区空调

图 7- 21 车型信息配置表

可一键跳转智能客服获得解答。



图 7- 22 跳转智能客服

并且可以对该车型进行打分及评价，生成的数据会影响个性化推荐的结果。



图 7- 23 车型打分评价界面

选中的车型可根据不同的场景获得智能化的推荐结果，例如当用户选择享界 S9，问界 M7，小米 YU7，五菱宏光 MINIEV，选择不同的场景会获得相应的推荐。

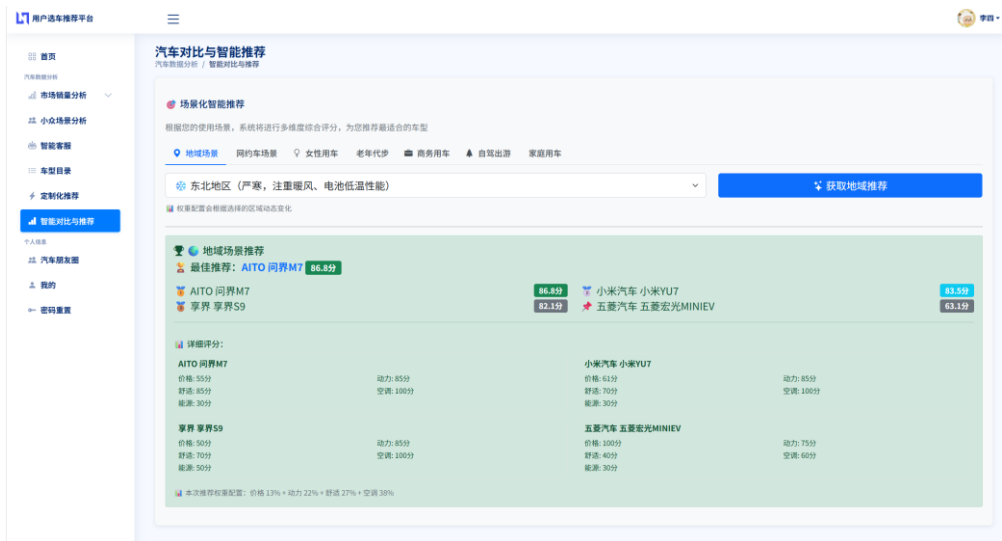


图 7-24 场景化智能推荐

用户亦可自行分配权重以获得更准确更个性化的车型推荐。

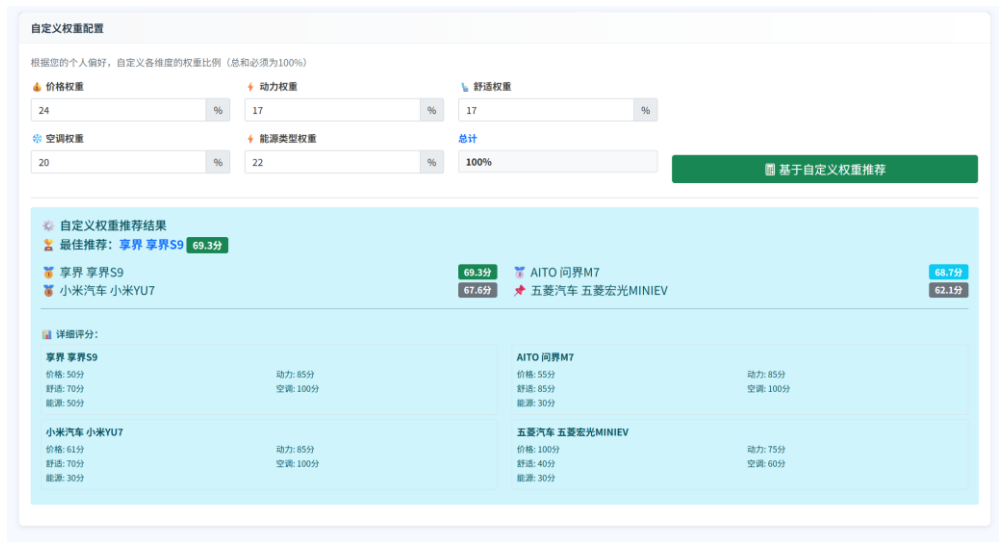


图 7-25 自定义权重推荐

本栏目也会设置表格对比，有优势的车型底部信息会有颜色标注。



图 7- 26 车型对比表

设置五张柱状图进行展示，用户可更直观的查看车型配置差异。



图 7- 27 车型对比柱状图

7.6 数据采集

该栏目设置了基础数据采集，条件数据采集和定时数据采集。

(1) 基础数据采集：管理员可选择采集数量一键点击采取数据。

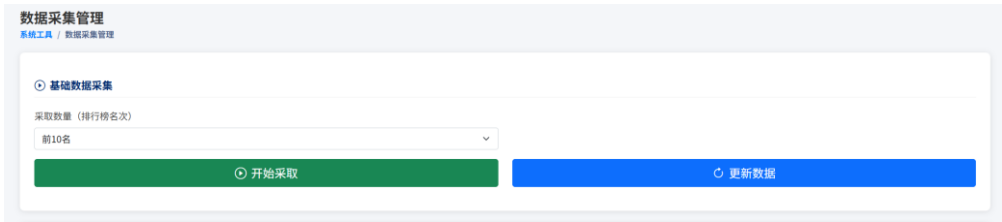


图 7-28 基础数据采集

(2) 条件数据采集：管理员可勾选不同条件如品牌，车型，能源类型，进行条件采集。



图 7-29 条件数据采集

(3) 定时数据采集：管理员可设置定时采集任务，到点系统会自动进行采集数据。



图 7-30 定时数据采集

数据采集过程中设置采集日志可视化，管理员可随时查看采集进度。

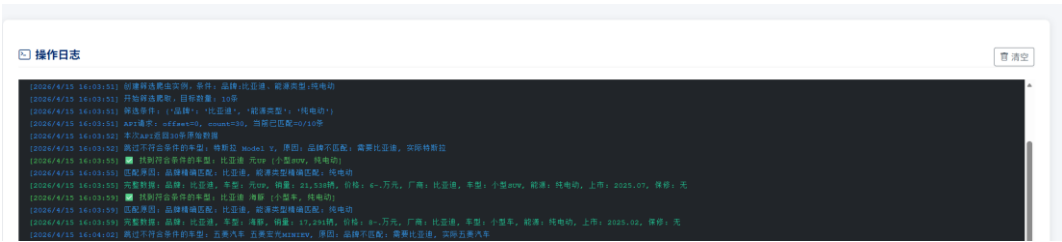


图 7-31 数据采集日志

7.7 管理员模块

(1) 管理员可对用户信息、汽车信息，汽车评价进行增删改。



图 7- 32 信息管理-用户管理

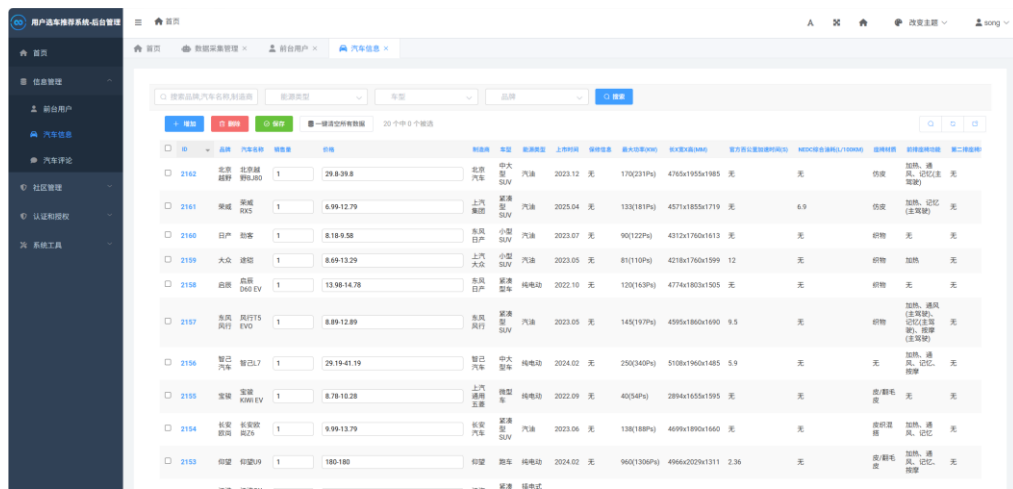


图 7- 33 信息管理-汽车信息

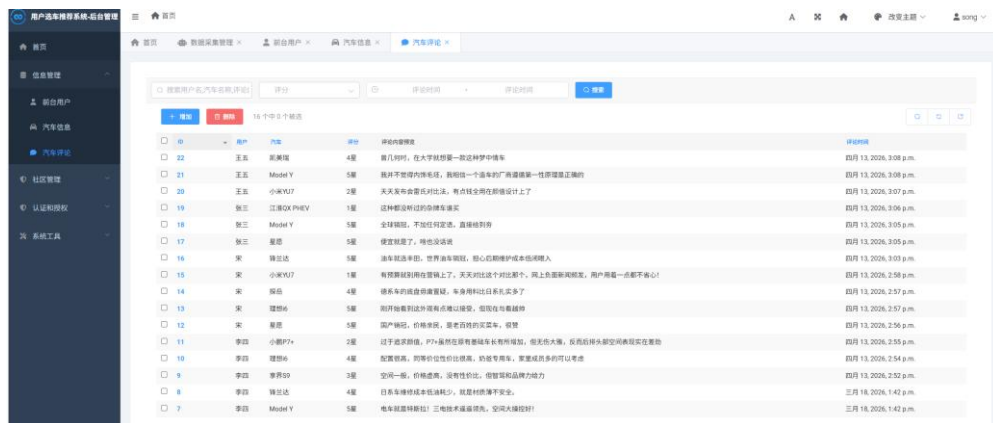


图 7- 34 信息管理-汽车评论

(2) 管理员可以对朋友圈信息、违规词信息进行增删改。



图 7- 35 社区管理-朋友圈管理

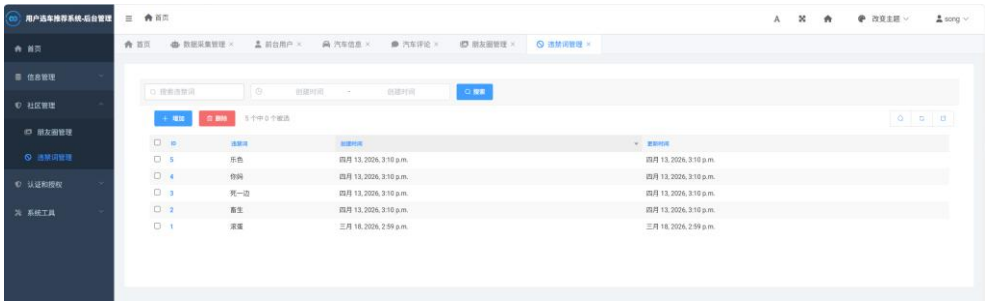


图 7- 36 社区管理-违禁词管理

(3) 最高权限管理员可增删管理员账户，组的管理和为管理员配置相应组和权限。

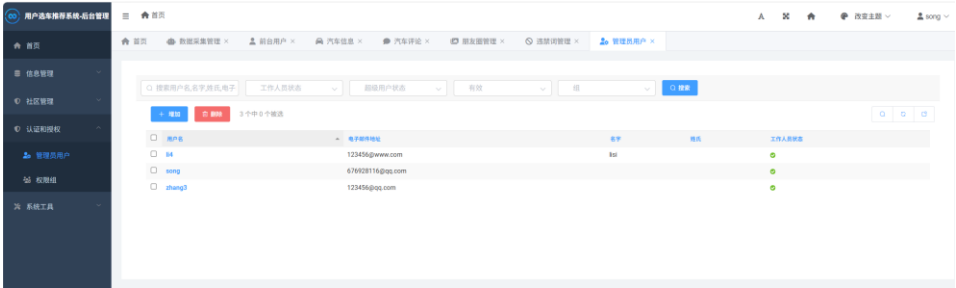


图 7- 37 认证与授权-管理员用户

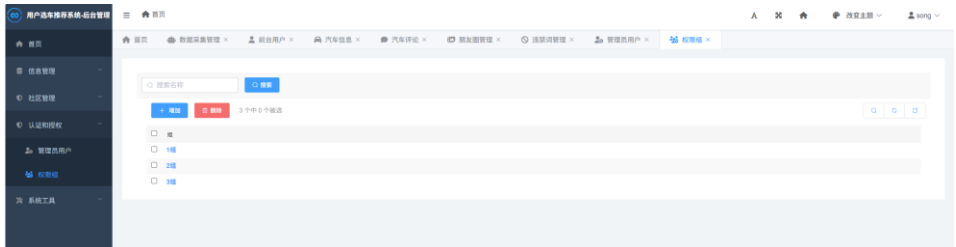


图 7- 38 认证与授权-组

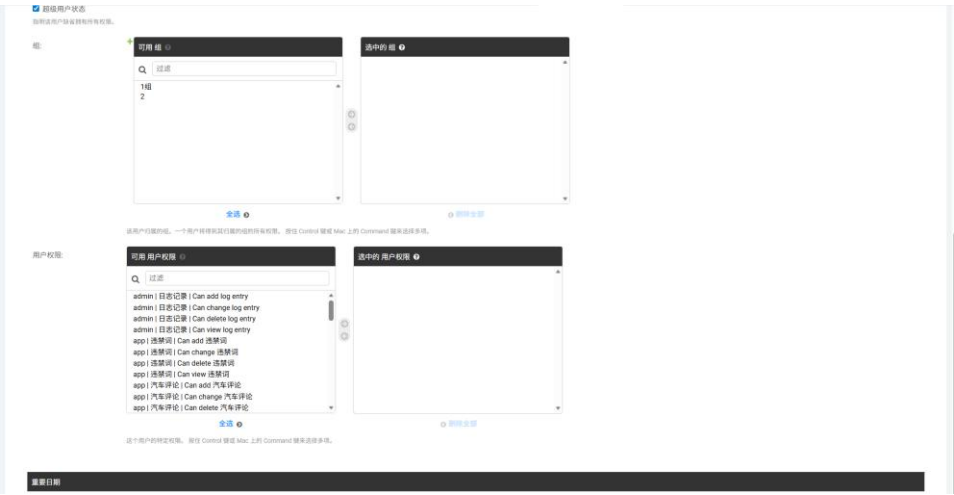


图 7-39 权限分配

参考文献

- [1] 徐畅, 赵萌, 孟鑫宇, 等. 考虑期望强度与属性补偿关系的新能源汽车在线推荐方法[J]. 管理学报, 2024, 21(7): 1056-1066.
- [2] 张胜根, 李斌, 易侃. 对标车型性能数据库[DS/OL]. 北京: 国家基础学科公共科学数据中心, 2021.
- [3] Django Software Foundation. Django[EB/OL]. (2025-03-01)[2026-04-20]. <https://www.djangoproject.com>.
- [4] ZHU Q, et al. Research on Configuration Price Database Recommendation System Based on User Collaborative Filtering Algorithm[C]//2024 IEEE 6th International Conference on Power, Intelligent Computing and Systems (ICPICS). Shenyang: IEEE, 2024: 1272-1279.
- [5] 罗刚. Scrapy 网络爬虫开发实战[M]. 北京: 清华大学出版社, 2024.
- [6] Apache ECharts 官方文档. ECharts[EB/OL]. (2025-03-01)[2026-04-20]. <https://echarts.apache.org>.
- [7] Bootstrap 官方文档. Bootstrap[EB/OL]. (2024-08-01)[2026-04-20]. <https://getbootstrap.com>.
- [8] BEN F. MySQL 必知必会[M]. 刘晓霞, 钟鸣, 译. 北京: 人民邮电出版社, 2009.
- [9] 高瞻, 严汝康, 向郑涛, 等. 车辆轨迹数据可视化分析系统的设计与实现[J]. 湖北汽车工业学院学报, 2019, 33(1): 42-46.
- [10] 广州市交通运输局. 2025 年 8 月网约车行业运营分析报告[R]. 广州: 广州市交通运输局, 2025.
- [11] 腾易研究院. 中国购车家庭财富洞察报告之资产负债表(2025 版)[R]. 北京: 腾易科技, 2025.
- [12] 中国旅游车船协会. 中国自驾车、旅居车与露营旅游发展报告

(2024-2025)[R]. 北京: 中国旅游车船协会, 2025.

致谢

行文至此，我的本科生涯宣告结束，首先要感谢自己，大学四年经历了很多困难与挑战，是靠自己的努力和坚强一步步走来，其次感谢我的家人朋友，是他们在背后默默支持和鼓励我前进，最后感谢我的导师张晓刚，在我的毕业设计和论文写作上给了很多建议指导。